

Forecasting Mumbai traffic — model report

Can the next hour of Mumbai traffic be predicted from the data this project collected? Short answer: **yes, and an LSTM does it best** — but only once it is compared against baselines that are genuinely hard to beat.

Generated by `ml/train.py` and `ml/report_ml.py`.

The task

Given a monitoring point's last 3 hours of speed ratio, predict that point's ratio **30, 60 and 90 minutes ahead**. One step is 30 minutes, matching the collection cadence.

Trained on era B: **50 points, 868 time steps, ~33,700 training samples** (all points pooled into one model).

Why the split matters

Train on the earliest 80% of time steps, test on the latest 20%. Concretely: train ends 5 Sept 06:00, test covers 5–8 Sept. A random split would let the model see the future and every score below would be meaningless.

An honest note on scale

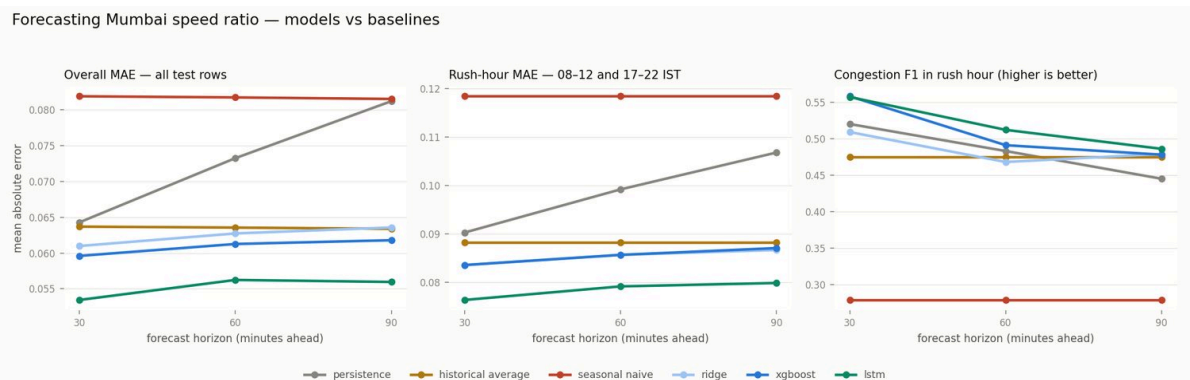
The standard benchmarks in this field — METR-LA and PEMS-BAY — carry 34,000+ time steps from 200+ sensors over 4–5 months. This dataset has **868 time steps**, roughly 40× less history. Spatio-temporal graph networks (DCRNN, Graph WaveNet) are built for that larger regime and would overfit here, so the model set is deliberately kept to methods that suit the data volume.

Baselines, and why they are not a formality

Traffic is highly autocorrelated: 30 minutes from now usually looks like now. That makes **persistence** a strong baseline, and plenty of published models fail to beat it at short horizons. Three baselines are used:

baseline	rule
persistence	the value stays where it is
historical average	that point, that hour, weekday vs weekend (computed on train only)
seasonal naive	the same clock time yesterday

Results



Overall error (lower is better)

model	MAE 30 min	MAE 60 min	MAE 90 min
persistence	0.0643	0.0733	0.0812
historical average	0.0637	0.0636	0.0634
seasonal naive	0.0819	0.0817	0.0815
ridge	0.0610	0.0627	0.0636
xgboost	0.0596	0.0613	0.0618
lstm	0.0534	0.0562	0.0559

The LSTM wins at every horizon, and — importantly — **its margin grows with horizon**: +17% better than persistence at 30 minutes, +23% at 60, +31% at 90. That is the expected shape: persistence decays as you look further ahead, a learned model holds up.

But overall MAE flatters everyone

Only **3% of test targets are congested**. Most rows are a free-flowing road at night, where predicting "1.00 stays 1.00" is trivial. The honest test is rush hour and already-slow roads:

model	rush MAE 30	rush MAE 60	rush MAE 90
persistence	0.0903	0.0992	0.1068
historical average	0.0883	0.0883	0.0883
seasonal naive	0.1184	0.1184	0.1184
ridge	0.0836	0.0857	0.0867
xgboost	0.0836	0.0857	0.0871
lstm	0.0764	0.0792	0.0799

The ranking survives. LSTM rush-hour MAE is 0.0764 / 0.0792 / 0.0799 against persistence at 0.0903 / 0.0992 / 0.1068 — a 15% to 25% improvement.

Calling the jam

Regression error is not what a commuter cares about. Treating "ratio below 0.5" as a congestion alarm:

model	F1 30 min	F1 60 min	F1 90 min
persistence	0.520	0.483	0.445
historical average	0.475	0.475	0.475
seasonal naive	0.279	0.279	0.279
ridge	0.509	0.468	0.479
xgboost	0.558	0.491	0.478
lstm	0.557	0.512	0.486

Around **0.5 F1** — useful, not solved. The models are better at precision than recall: when they call a jam they are usually right, but they miss roughly half of them. Sudden-onset jams are exactly what 30-minute sampling cannot see coming.

Three findings worth keeping

1. **The LSTM's advantage is real but modest, and it grows with horizon.** At 30 minutes persistence is nearly as good; by 90 minutes the gap is clear. Anyone claiming a large short-horizon win over persistence should be asked what their baseline was.
2. **Historical average is remarkably strong.** It ignores current conditions entirely, yet beats persistence from 60 minutes out. Mumbai traffic is highly routine — the clock predicts congestion better than the present does, once you look far enough ahead.
3. **On already-slow roads, XGBoost edges out the LSTM at 60 and 90 minutes** (0.1176 vs 0.1197). The LSTM wins on average; it is not uniformly better.

Limitations

- **868 time steps.** Enough for these models, not enough for spatio-temporal graph networks. More collection is the single biggest available improvement.
- **One test window** (5–8 Sept). No repeated cross-validation across folds, so treat the margins as indicative rather than precise.
- **Spatial structure is barely used.** Only a `neighbour_mean` feature; the adjacency graph built in `adjacency.csv` is not exploited by a real graph model, because the data volume does not support one.
- **The LSTM is optimised for MAE** (L1 loss), so it wins on MAE while XGBoost sometimes wins on RMSE. That is a consequence of the objective, not evidence of a better model.
- Speed ratio depends on TomTom's modelled free-flow baseline, not a measured overnight speed.

Reproducing

```
python ml/dataset.py      # dataset shapes and feature list
python ml/train.py        # trains everything, writes ml/results/
python ml/report_ml.py    # this report and its figure
```